

# Plan de Implementación de Actualización en Tiempo Real

---

React + NestJS + PostgreSQL + Prisma + Socket.IO

---

## Objetivo

---

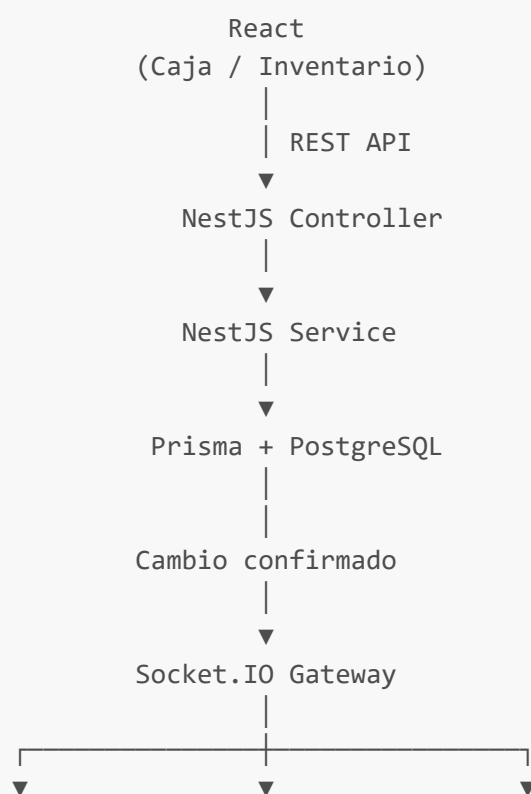
Implementar un sistema de comunicación en tiempo real para que todos los usuarios conectados visualicen inmediatamente cualquier cambio realizado en el sistema, sin necesidad de actualizar la página.

El sistema deberá sincronizar:

- Stock
  - Productos
  - Ventas
  - Compras
  - Traslados
  - Dashboard
  - Notificaciones
  - Estado de usuarios
- 

## Arquitectura General

---



|            |                  |                 |
|------------|------------------|-----------------|
| React Caja | React Inventario | React Dashboard |
| ▼          | ▼                | ▼               |
| Celular    | Administrador    | Supervisor      |

## Tecnologías

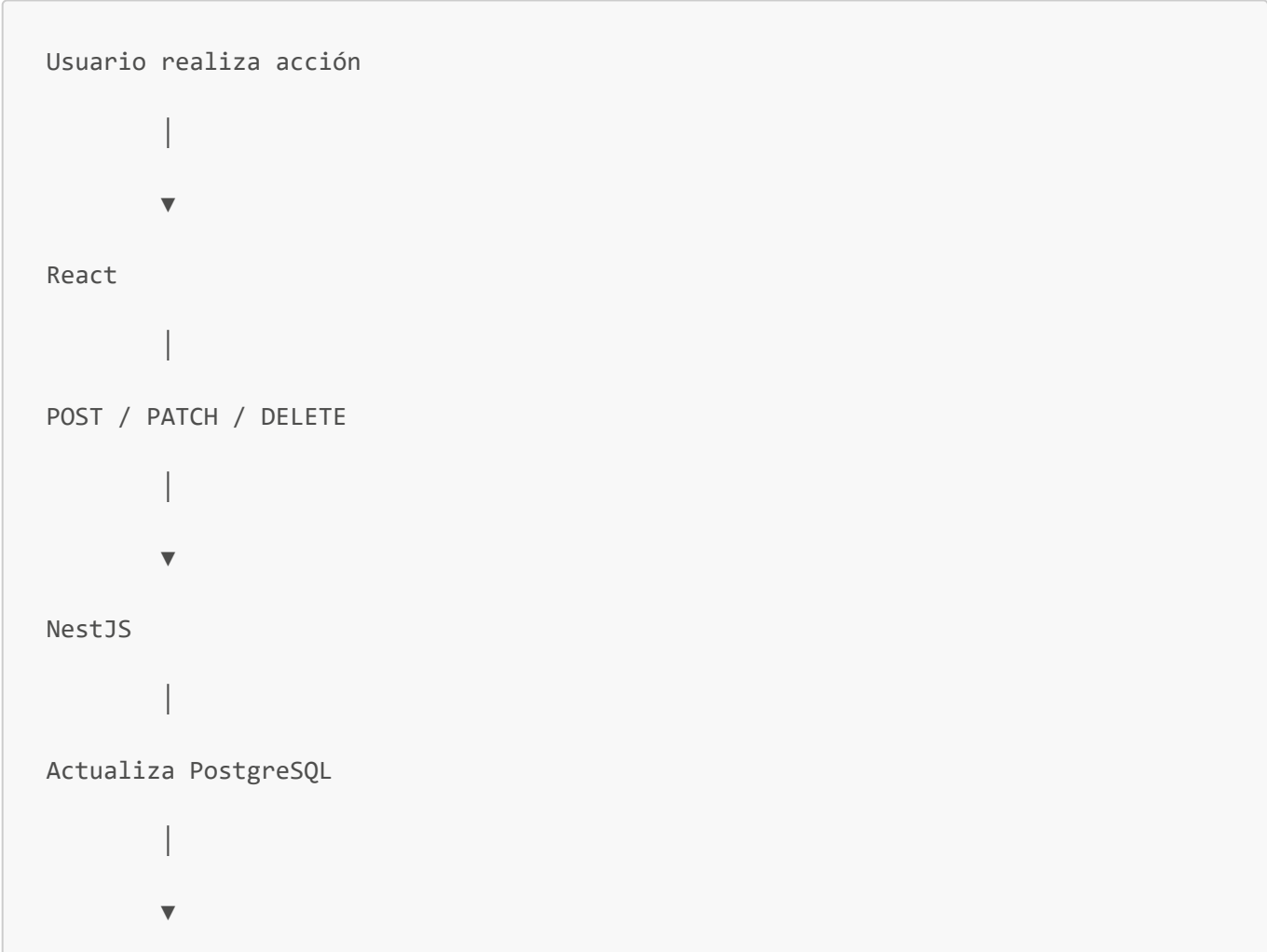
### Frontend

- React
- TypeScript
- Socket.IO Client

### Backend

- NestJS
- Socket.IO
- Prisma
- PostgreSQL

## Flujo General



Prisma confirma cambios

|

▼

Socket.IO emite evento

|

|————→ Caja 1

|————→ Caja 2

|————→ Dashboard

|————→ Inventario

|————→ Celular

Todos reciben la actualización en menos de un segundo.

---

## Organización del Proyecto

---

backend/

src/

websocket/

websocket.module.ts

websocket.gateway.ts

inventory/

sales/

purchases/

transfer/

dashboard/

frontend/

src/

```
socket/  
  socket.ts  
  
hooks/  
  useSocket.ts  
  
providers/  
  SocketProvider.tsx
```

---

## Fase 1

---

### Configurar Socket.IO

#### Objetivos

- Instalar Socket.IO
- Crear Gateway
- Permitir conexiones
- Manejar desconexiones

#### Resultado esperado

Todos los clientes pueden conectarse al servidor.

---

## Fase 2

---

### Conexión desde React

#### Crear

```
socket.ts
```

#### Responsabilidades

- Conectarse automáticamente
- Reconectarse
- Detectar pérdida de conexión
- Manejar autenticación

#### Resultado

Todos los usuarios permanecen conectados.

---

## Fase 3

---

### Crear Eventos

Cada operación importante generará un evento.

Eventos mínimos

```
producto.created  
producto.updated  
producto.deleted  
stock.updated  
venta.created  
venta.cancelled  
compra.created  
traslado.created  
dashboard.updated  
notification.created
```

---

## Fase 4

---

### Inventario en Tiempo Real

Cada vez que cambie un producto

```
Actualizar Stock  
Actualizar Precio  
Actualizar Lote  
Actualizar Fecha de Vencimiento  
Eliminar Producto  
Crear Producto
```

---

NestJS emitirá

```
stock.updated
```

Payload

```
{  
  "productId": "uuid",  
  "stock": 24  
}
```

React actualizará únicamente ese producto.

---

## Fase 5

---

### Ventas en Tiempo Real

Cuando una venta termina

NestJS

1. Guarda venta
2. Reduce stock
3. Actualiza estadísticas
4. Envía evento

Eventos

```
venta.created  
  
stock.updated  
  
dashboard.updated
```

Todos los clientes actualizan automáticamente.

---

## Fase 6

---

### Compras

Cuando llegue mercadería

Registrar Compra

Actualizar Stock

Actualizar Costos

Actualizar Dashboard

## Eventos

compra.created

stock.updated

dashboard.updated

---

# Fase 7

---

## Traslado entre almacenes

### Proceso

Origen

↓

Descontar stock

↓

Destino

↓

Agregar stock

↓

Emitir evento

## Eventos

```
stock.updated  
  
traslado.created
```

Todas las sucursales reciben el cambio.

---

## Fase 8

---

### Dashboard

Cada venta modifica

```
Ventas del día  
  
Productos vendidos  
  
Productos críticos  
  
Ganancias  
  
Número de ventas
```

Evento

```
dashboard.updated
```

El Dashboard nunca necesita recargarse.

---

## Fase 9

---

### Notificaciones

Ejemplos

```
Producto agotado  
  
Stock mínimo  
  
Compra registrada  
  
Venta anulada
```



Nuevo usuario

Traslado realizado

Evento

notification.created

---

## Fase 10

---

### Usuarios Conectados

Cada login

user.connected

Cada cierre

user.disconnected

El administrador puede visualizar

Caja 1

Caja 2

Inventario

Administrador

Supervisor

---

## Fase 11

---

### Salas (Rooms)

No todos los usuarios deben recibir todos los eventos.

Ejemplo

Sucursal Lima

Sucursal Ica

Sucursal Arequipa

Cada usuario entra únicamente en su sala.

join(room)

Después

emit(room)

Solo los usuarios de esa sucursal reciben el evento.

---

## Fase 12

---

### Roles

Administrador

Recibe

Todo

Inventario

Recibe

Productos

Stock

Compras

Caja

Recibe

Ventas

Stock

Dashboard

Recibe

Dashboard

Ventas

Ganancias

---

## Fase 13

---

### Reconexión

Si Internet falla

Socket desconectado

↓

Reconexión automática

↓

Sincronizar información

El usuario no pierde la sesión.

---

## Fase 14

---

### Optimización

Evitar enviar toda la tabla.

Incorrecto

5000 productos

---

Correcto

Enviar únicamente

```
{  
  "productoId": "A12",  
  "stock": 8  
}
```

React modifica únicamente ese elemento.

---

## Fase 15

---

### Sincronización Local

Cuando un evento llega

```
Socket  
↓  
Context  
↓  
Redux o Zustand  
↓  
React  
↓  
Componente
```

No es necesario volver a consultar la API.

---

## Fase 16

---

### Manejo de Conflictos

Dos usuarios venden el mismo medicamento.

Proceso

Usuario A

↓

Servidor

↓

Actualizar Stock

↓

Emitir Evento

↓

Usuario B recibe nuevo stock

Nunca se modifica el stock desde React.

Toda la lógica permanece en el servidor.

---

## Fase 17

---

### Seguridad

Autenticar Socket mediante JWT

Validar usuario

Validar sucursal

Validar permisos

Cerrar conexión si el token expira

---

## Eventos Finales

---

### Productos

producto.created

producto.updated

producto.deleted

---

## Inventario

stock.updated

stock.minimum

stock.out

---

## Ventas

venta.created

venta.cancelled

---

## Compras

compra.created

---

## Traslados

traslado.created

traslado.completed

---

## Dashboard

dashboard.updated

---

## Usuarios

user.connected

user.disconnected

```
session.closed
```

## Notificaciones

```
notification.created
```

## Flujo Completo



Caja 1 recibe cambio

Caja 2 recibe cambio

Inventario recibe cambio

Dashboard recibe cambio

Administrador recibe cambio

Celular recibe cambio

---

React actualiza únicamente el producto afectado

---

## Beneficios

---

- Actualización en tiempo real.
- Sin necesidad de recargar la página.
- Menor consumo de recursos.
- Escalable para múltiples sucursales.
- Compatible con dispositivos móviles.
- Comunicación bidireccional cliente-servidor.
- Preparado para notificaciones y monitoreo en vivo.
- Base sólida para futuras funciones como chat interno, alertas de stock, monitoreo de cajas y seguimiento de pedidos.

---

## Próximas Implementaciones Recomendadas

---

1. Gateway de Socket.IO con autenticación JWT.
2. Gestión de salas por sucursal y rol.
3. Servicio centralizado para emitir eventos ([RealtimeService](#)).
4. Contexto de React ([SocketProvider](#)) para compartir la conexión.
5. Hooks personalizados ([useStock](#), [useNotifications](#), [useDashboard](#)).
6. Integración con Zustand o Redux para actualizar el estado global.
7. Reconexión automática y sincronización al recuperar la conexión.
8. Panel de monitoreo de usuarios conectados y sesiones activas.